

Technical Documentation

Customer Segmentation Project — For Developers

1. System Architecture

The application follows a layered architecture: a data layer (raw + processed CSVs), a shared logic layer (src/ — preprocessing, feature engineering, prediction), a saved-model layer (models/ — StandardScaler + K-Means, pickled), and a presentation layer (app/ — Streamlit dashboard). The same src/ modules are used both to prepare training data and to score new customers submitted through the dashboard, guaranteeing training/inference consistency.

2. Project Structure

```
customer-segmentation-project/ |— app/           # Streamlit dashboard (app.py,
dashboard.py, components/) |— data/raw/         # Original marketing_campaign.csv |—
data/processed/           # Final engineered + segmented dataset |— models/           #
scaler.pkl, kmeans_model.pkl, production_pipeline.pkl |— src/           #
preprocessing.py, feature_engineering.py, prediction.py |— tests/         # pytest
suite (40 tests) |— reports/           # QA reports (Excel) |— notebooks/       #
Key Jupyter notebooks |— documentation/       # This documentation set |— presentation/
# Slides + demo script |— requirements.txt |— README.md
```

3. Environment Requirements

Requirement	Version / Detail
Python	3.10 or newer (developed on 3.12)
OS	Windows, macOS, or Linux — no OS-specific dependencies
Memory	2 GB+ recommended (dataset and model are small, <5 MB combined)

4. Dependencies

All dependencies are pinned in requirements.txt: streamlit, pandas, numpy, scikit-learn, plotly, joblib, openpyxl, matplotlib, seaborn, pytest.

5. Installation Instructions

```
git clone <repository-url> cd customer-segmentation-project pip install -r
requirements.txt
```

6. Model Loading

src/prediction.py's SegmentationPipeline class loads models/production_pipeline.pkl on first use (via joblib.load), which bundles the fitted StandardScaler, the trained K-Means model, the exact training feature list (in order), and the random_state used.

7. Data Pipeline

Raw CSV -> src/preprocessing.load_and_clean() -> src/feature_engineering.create_features() -> feature selection to the 12 clustering features -> StandardScaler.transform() -> KMeans.predict(). The dashboard's data_loader.py loads the already-processed, already-segmented CSV for display; the prediction form runs new customers through the full live pipeline.

8. Application Execution

```
streamlit run app/app.py # Dashboard opens at http://localhost:8501
```

9. Configuration

No environment variables or secrets are required. File paths are resolved relative to the project root via `Path(__file__).resolve().parent` chains, so the app can be run from any working directory.

10. Troubleshooting

Symptom	Likely Cause	Fix
ModuleNotFoundError: streamlit	Dependencies not installed	<code>pip install -r requirements.txt</code>
FileNotFoundError on data/processed CSV	Running from wrong directory or missing data file	Run from project root; confirm <code>data/processed/customer_segments_with_strategy.csv</code> exists
Model load error	<code>models/production_pipeline.pkl</code> missing or corrupted	Re-copy from models/ backup or retrain via Module 06 pipeline
Port already in use	Another Streamlit instance running	<code>streamlit run app/app.py --server.port 8502</code>